SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:40**

Annexure A

Industry Problem Solving Task

Code of Conduct:

*I **Niranjana(192521370)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Niranjana A

Annexure A

Industry Problem Solving Task

OBJECT-ORIENTED DESIGN, LAYERED ARCHITECTURE AND DESIGN PATTERNS

1. Smart Banking and Financial Management System

Introduction

- The Smart Banking and Financial Management System is an object-oriented application developed to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The system follows a layered architecture consisting of the Presentation Layer, Service Layer, Domain Layer, and Data Access Layer. An Object-Oriented Database Management System (OODBMS) is used to store and retrieve persistent banking objects.
- The system uses the Factory Pattern for creating different account types, the Strategy Pattern for loan-interest and payment calculation policies, the Observer Pattern for transaction and fraud alerts, and the Repository Pattern for communication with persistent banking objects.
- Object-Oriented Principles
- The system applies important object-oriented principles such as encapsulation, inheritance, polymorphism, and abstraction.
- The main classes of the system include Customer, Account, SavingsAccount, CurrentAccount, Loan, Transaction, FundTransfer, and BillPayment.
- Encapsulation protects sensitive information such as account balance, customer details, and transaction information by restricting direct access to data.
- Inheritance can be used to create SavingsAccount and CurrentAccount as subclasses of the Account class. Common properties and methods can be defined in the Account class and reused by the subclasses.

- Polymorphism allows different account types to provide their own implementation of operations such as interest calculation and withdrawal.
- Abstraction hides complex banking operations and database implementation details from the users and other components.

Layered Architecture

The banking system can be organized into the following layers:

PresentationLayer

↓

ServiceLayer

↓

Domain Layer

↓

Data Access Layer

↓

OODBMS

Presentation Layer

The Presentation Layer provides the user interface for customers and bank employees. It handles operations such as account registration, fund transfer, loan application, bill payment, and viewing transaction history.

Service Layer

The Service Layer contains the main business logic of the application. It can contain services such as AccountService, LoanService, FundTransferService, PaymentService, and TransactionService.

For example, FundTransferService validates the account information, checks the balance, performs the transfer, and updates the transaction history.

Domain Layer

The Domain Layer contains the core banking objects such as Customer, Account, Loan, Transaction, and Payment. These objects represent the real-world entities of the banking system.

Data Access Layer

The Data Access Layer is responsible for communicating with the OODBMS. The Repository Pattern provides methods for saving, updating, deleting, and retrieving persistent banking objects.

Design Patterns

Factory Pattern

The Factory Pattern is used to create different account types.

AccountFactory

↓

SavingsAccount

CurrentAccount

BusinessAccount

The client does not need to directly create concrete account objects. The factory determines which account object should be created.

Strategy Pattern

The Strategy Pattern is used for loan-interest calculation and payment calculation.

CalculationStrategy

↓

LoanInterestStrategy

PaymentStrategy

Different strategies can be selected without changing the main banking service. For example, different interest policies can be implemented as separate strategy classes.

Observer Pattern

The Observer Pattern is used for transaction and fraud alerts.

When an important transaction occurs, registered observers such as customers and fraud-monitoring services are notified.

Transaction

↓

Observers

↓

Customer / Fraud Alert Service

This allows real-time notifications without tightly coupling the transaction class with notification components.

Repository Pattern

The Repository Pattern provides an abstraction between the service layer and the OODBMS. Banking services communicate with repositories instead of directly accessing the database.

For example:

FundTransferService

↓

AccountRepository

↓

OODBMS

Communication with OODBMS

When a customer performs a banking operation, the request first reaches the Presentation Layer. The Presentation Layer sends the request to the appropriate Service Layer. The service performs the required business validation and interacts with the Domain objects.

The service then uses a Repository to store or retrieve persistent objects from the OODBMS.

For example, during a fund transfer:

Customer

↓

PresentationLayer

↓

FundTransferService

↓

AccountRepository

↓

OODBMS

The OODBMS stores persistent objects such as Customer, Account, Loan, and Transaction objects. When required, the Repository retrieves these objects and provides them to the Service Layer.

Benefits

The object-oriented design improves security through encapsulation and controlled access to sensitive banking information.

The layered architecture provides separation of concerns, making the system easier to understand, test, and maintain.

The Factory Pattern makes the system flexible when new account types are introduced.

The Strategy Pattern allows new interest and payment policies to be added without modifying existing business logic.

The Observer Pattern supports real-time transaction and fraud notifications.

The Repository Pattern separates database operations from business logic.

Therefore, the system becomes scalable, secure, flexible, reusable, and maintainable. Future services such as investment management and digital wallets can be added without major changes to the existing architecture.

2. Smart Retail and Inventory Management System

Introduction

- The Smart Retail and Inventory Management System is an object-oriented application designed to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered architecture consisting of Presentation, Business, Domain, and Data Access layers connected to an OODBMS.
- The system uses the MVC Pattern for user interface management, the Factory Pattern for creating different product categories, the Observer Pattern for low-stock notifications, and the DAO Pattern for communication with persistent inventory objects.
- Object-Oriented Principles
- The major classes of the system include Product, Supplier, Customer, PurchaseOrder, SalesTransaction, Warehouse, Inventory, and StockItem.
- Encapsulation protects product prices, stock quantities, customer details, and supplier information.
- Inheritance can be used to represent different product categories. For example, ElectronicsProduct, GroceryProduct, and ClothingProduct can inherit from a common Product class.
- Polymorphism allows different product categories to implement category-specific operations.
- Abstraction hides complex inventory and database operations from the user interface.
- These principles improve code reuse, flexibility, and maintainability.

Layered Architecture

PresentationLayer

↓

BusinessLayer

↓

DomainLayer

↓

DataAccessLayer

↓

OODBMS

Presentation Layer

The Presentation Layer provides interfaces for product management, sales, purchasing, warehouse management, and stock monitoring.

The MVC Pattern can be used in this layer.

Model ↔ Controller ↔ View

The View displays information to users. The Controller receives user requests and communicates with the Business Layer. The Model represents the application data.

Business Layer

The Business Layer contains business rules related to inventory, sales, purchasing, and stock management.

Examples include InventoryService, SalesService, PurchaseService, and SupplierService.

Domain Layer

The Domain Layer contains Product, Inventory, Supplier, Customer, PurchaseOrder, and SalesTransaction objects.

Data Access Layer

The Data Access Layer contains DAO classes that communicate with the OODBMS.

Examples include ProductDAO, InventoryDAO, SupplierDAO, and SalesDAO.

Design Patterns

MVC Pattern

The MVC Pattern separates the user interface into Model, View, and Controller components.

This prevents user-interface code from becoming tightly coupled with business logic.

Factory Pattern

The Factory Pattern is used to create different product categories.

ProductFactory

↓

Electronics

Grocery

Clothing

If a new product category is added, a new class can be created without changing the existing client code significantly.

Observer Pattern

The Observer Pattern is used to notify managers when inventory reaches a minimum threshold.

Inventory

↓

Observer

↓

Manager / Warehouse Staff

Whenever the stock level becomes lower than the defined threshold, the registered observers receive a notification.

DAO Pattern

The DAO Pattern separates database operations from business logic.

InventoryService

↓

InventoryDAO

↓

OODBMS

The DAO performs operations such as storing, updating, deleting, and retrieving inventory objects.

Communication with OODBMS

When a user updates stock, the request is received by the Controller. The Controller communicates with the Business Layer. The Business Layer performs the required business operation and sends the request to the appropriate DAO.

For example:

User

↓

Controller

↓

InventoryService

↓

InventoryDAO

↓

OODBMS

The OODBMS stores persistent objects such as Product, Inventory, Supplier, Warehouse, and **PurchaseOrder.**

When stock information is required, the DAO retrieves the persistent inventory objects from the OODBMS and provides them to the Business Layer.

Benefits

- Object-oriented principles provide reusable and modular classes.
- The MVC Pattern separates user-interface logic from business logic.
- The Factory Pattern makes it easier to introduce new product categories.
- The Observer Pattern provides automatic low-stock notifications.
- The DAO Pattern reduces database code duplication and improves persistence management.
- The layered architecture improves maintainability because each layer has a specific responsibility.
- The system can be extended in the future to support automated stock prediction and robotic warehouse management.
- Therefore, the combination of OOP, layered architecture, and design patterns provides flexibility, scalability, reusability, and maintainability.

3. Smart Hotel and Hospitality Management System

Introduction

- The Smart Hotel Management System is an object-oriented application used to manage guest registration, room reservations, check-in, check-out, payments, room services, and customer feedback. The system follows a layered architecture consisting of Presentation, Service, Domain, and Data Access layers connected to an OODBMS.
- The Strategy Pattern is used for pricing and discount policies, the Factory Pattern is used to create room and service objects, the Observer Pattern is used for reservation and service notifications, and the Repository Pattern is used for communication with persistent hotel objects.

Object-Oriented Principles

The main classes include Guest, Room, Reservation, Payment, RoomService, Feedback, and Hotel.

Inheritance can be used for different room types.

Room

↓

SingleRoom

DoubleRoom

Suite

Encapsulation protects guest information, reservation details, room availability, and payment information.

Polymorphism allows different room types to implement different pricing or service operations.

Abstraction hides the internal implementation of reservation processing and database communication.

Layered Architecture

Presentation Layer

↓

Service Layer

↓

Domain Layer

↓

Repository Layer

↓

OODBMS

Presentation Layer

The Presentation Layer provides screens for guest registration, room booking, check-in, check-out, payment, and room service.

Service Layer

The Service Layer contains business services such as GuestService, ReservationService, PaymentService, and RoomService.

For example, ReservationService checks room availability, creates a reservation, and updates the reservation status.

Domain Layer

The Domain Layer contains Guest, Room, Reservation, Payment, RoomService, and Feedback objects.

Repository Layer

The Repository Layer communicates with the OODBMS and manages persistent hotel objects. Examples include GuestRepository, RoomRepository, ReservationRepository, and PaymentRepository.

Design Patterns

Strategy Pattern

The Strategy Pattern is used to implement different pricing and discount policies.

PricingStrategy

↓

StandardPricing

SeasonalPricing

DiscountPricing

A pricing strategy can be changed without modifying the ReservationService.

Factory Pattern

The Factory Pattern is used to create room and service objects.

RoomFactory

↓

SingleRoom

DoubleRoom

Suite

This centralizes object creation and reduces duplication.

Observer Pattern

The Observer Pattern is used to send reservation and room-service notifications.

Reservation

↓

Observers

↓

Guest / Hotel Staff

When a reservation is confirmed or modified, registered observers can receive notifications.

Repository Pattern

The Repository Pattern provides a simple interface for storing and retrieving persistent hotel objects.

ReservationService

↓

ReservationRepository

↓

OODBMS

Reservation Processing and OODBMS

When a guest makes a reservation, the request first reaches the Presentation Layer.

Guest

↓

ReservationScreen

↓

ReservationService

↓

RoomRepository

↓

OODBMS

The repository retrieves available Room objects from the OODBMS. After selecting a room, the ReservationService creates a Reservation object and stores it through the ReservationRepository.

The same process can be used for guest details, payments, and room-service information.

Benefits

- The Strategy Pattern provides flexibility in changing pricing and discount policies.
- The Factory Pattern provides reusable object creation mechanisms.
- The Observer Pattern provides automatic reservation and service notifications.
- The Repository Pattern separates business logic from persistence operations.
- Layered architecture improves maintainability and reduces coupling.
- The system can easily be extended to support smart-room automation and personalized guest services.
- Thus, the object-oriented design provides flexibility, reusability, scalability, and maintainability.

4. Smart Logistics and Delivery Management System

Introduction

- The Smart Logistics and Delivery Management System is designed to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system follows a layered object-oriented architecture consisting of Presentation, Business, Domain, and Data Access layers connected to an OODBMS.

- The Strategy Pattern is used for route selection, the Factory Pattern is used for shipment creation, the Observer Pattern is used for real-time delivery notifications, and the DAO Pattern manages persistent shipment and vehicle information.

Object-Oriented Principles

The main classes include Shipment, Vehicle, Warehouse, Delivery, Customer, Route, and Notification.

Inheritance can represent different shipment types.

Shipment

↓

StandardShipment

ExpressShipment

FragileShipment

Encapsulation protects shipment status, vehicle information, customer details, and delivery information.

Polymorphism allows different shipment types to apply different delivery rules.

Abstraction hides route optimization and persistence implementation details.

Layered Architecture

PresentationLayer

↓

BusinessLayer

↓

DomainLayer

↓

DAOLayer

↓

OODBMS

Presentation Layer

The Presentation Layer provides interfaces for shipment registration, warehouse management, vehicle allocation, and delivery tracking.

Business Layer

The Business Layer contains services such as ShipmentService, VehicleService, DeliveryService, RouteService, and NotificationService.

It handles shipment processing, vehicle allocation, route selection, and delivery tracking.

Domain Layer

The Domain Layer contains Shipment, Vehicle, Warehouse, Delivery, Customer, and Route objects.

Data Access Layer

The DAO Layer manages database operations.

Examples include ShipmentDAO, VehicleDAO, WarehouseDAO, and DeliveryDAO.

Design Patterns

Strategy Pattern

The Strategy Pattern is used to select an optimal delivery route.

RouteStrategy

↓

FastestRoute

ShortestRoute

LowestCostRoute

The system can select an appropriate route depending on business requirements.

Factory Pattern

The Factory Pattern creates different shipment types.

ShipmentFactory

↓

StandardShipment

ExpressShipment

FragileShipment

This centralizes object creation.

Observer Pattern

The Observer Pattern provides real-time delivery notifications.

Delivery

↓

Observers

↓

Customer / Manager

Whenever delivery status changes, registered observers receive notifications.

DAO Pattern

The DAO Pattern handles persistence operations.

DeliveryService

↓

DeliveryDAO

↓

OODBMS

The DAO stores and retrieves persistent shipment, vehicle, and delivery objects.

Communication with OODBMS

The layers communicate with the OODBMS through the DAO layer.

For example:

Customer

↓

PresentationLayer

↓

DeliveryService

↓

ShipmentDAO

↓

OODBMS

The OODBMS stores Shipment, Vehicle, Warehouse, Delivery, and Customer objects.

When tracking information is requested, the DAO retrieves the appropriate persistent objects and sends them to the Business Layer.

When the delivery status changes, the updated object is stored back in the OODBMS through the DAO.

Benefits

- The Strategy Pattern allows new route optimization algorithms to be introduced easily.
- The Factory Pattern makes the system flexible when new shipment types are added.
- The Observer Pattern supports real-time customer notifications.
- The DAO Pattern separates persistence logic from business logic.
- Layered architecture improves maintainability and reduces coupling.
- The system can be extended to support autonomous delivery vehicles and AI-based route optimization.
- Therefore, the architecture provides adaptability, scalability, flexibility, and maintainability.

5. Smart Energy Management System

Introduction

- The Smart Energy Management System is designed to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal

consumption. The system follows a layered object-oriented architecture consisting of Presentation, Business, Domain, and Data Access layers connected to an OODBMS.

- The Observer Pattern is used for consumption alerts, the Strategy Pattern is used for energy optimization algorithms, the Factory Pattern is used for creating smart-meter objects, and the Repository Pattern is used for managing persistent energy data.
- Object-Oriented Principles
- The major classes include SmartMeter, EnergyReading, Consumer, EnergyAnalyzer, Alert, and EnergyReport.

Different smart meters can inherit from a common SmartMeter class.

SmartMeter

↓

ResidentialMeter

IndustrialMeter

SolarMeter

Encapsulation protects energy consumption information and meter data.

Polymorphism allows different types of smart meters to implement their own reading or monitoring operations.

Abstraction hides complex energy analysis and optimization algorithms.

Layered Architecture

PresentationLayer

↓

BusinessLayer

↓

DomainLayer

↓

RepositoryLayer

↓

OODBMS

Presentation Layer

The Presentation Layer provides dashboards for displaying electricity consumption, energy reports, meter information, and alerts.

Business Layer

The Business Layer performs energy analysis, abnormal consumption detection, reporting, and energy optimization.

Examples include EnergyAnalysisService, MeterService, AlertService, and OptimizationService.

Domain Layer

The Domain Layer contains SmartMeter, EnergyReading, Consumer, Alert, and EnergyReport objects.

Repository Layer

The Repository Layer communicates with the OODBMS.

Examples include SmartMeterRepository, EnergyReadingRepository, and ConsumerRepository.

Design Patterns

Observer Pattern

The Observer Pattern is used to generate alerts when abnormal energy consumption is detected.

EnergyMonitor

↓

Observer

↙ ↘

Consumer Administrator

When abnormal consumption is detected, the registered observers are automatically notified.

Strategy Pattern

The Strategy Pattern supports different energy optimization algorithms.

OptimizationStrategy

↓

CostOptimization

PeakReduction

EnergySaving

Different algorithms can be selected depending on the energy-management requirement.

Factory Pattern

The Factory Pattern is used to create different smart-meter objects.

SmartMeterFactory

↓

ResidentialMeter

IndustrialMeter

SolarMeter

This reduces object-creation duplication.

Repository Pattern

The Repository Pattern manages persistent energy-related objects.

EnergyService

↓

EnergyRepository

↓

OODBMS

The Repository provides methods for saving, updating, retrieving, and deleting persistent energy objects.

Communication with OODBMS

The Presentation Layer sends user requests to the Business Layer. The Business Layer processes the request and communicates with Domain objects.

For storing energy readings:

SmartMeter

↓

EnergyService

↓

EnergyRepository

↓

OODBMS

The OODBMS stores persistent objects such as SmartMeter, EnergyReading, Consumer, Alert, and EnergyReport.

When the dashboard requests historical energy consumption, the Business Layer requests the required objects from the Repository. The Repository retrieves the objects from the OODBMS and returns them to the Business Layer.

Benefits

- The Observer Pattern provides automatic alerts for abnormal energy consumption.
- The Strategy Pattern makes energy optimization algorithms replaceable and extensible.
- The Factory Pattern simplifies the creation of different smart-meter objects.
- The Repository Pattern separates persistence operations from business logic.
- The layered architecture provides clear separation of responsibilities and improves maintainability.
- The system can be extended to support renewable-energy monitoring and AI-based consumption prediction.

- Therefore, the combination of object-oriented principles, layered architecture, and design patterns provides scalability, extensibility, flexibility, reusability, and maintainability.

CONCLUSION

- All five smart management systems demonstrate how object-oriented principles, layered architecture, and design patterns can be combined to develop scalable and maintainable software systems.
- Object-oriented principles such as encapsulation, inheritance, polymorphism, and abstraction help represent real-world entities as reusable software objects. Layered architecture separates presentation, business logic, domain objects, and database operations, thereby reducing coupling and improving maintainability.
- The Factory Pattern provides flexible object creation, the Strategy Pattern allows algorithms and policies to be changed easily, the Observer Pattern supports automatic notifications, and DAO or Repository patterns separate persistence operations from business logic.
- The OODBMS provides persistent storage for domain objects. The Data Access Layer acts as an intermediate layer between the application and the OODBMS, allowing objects to be stored, updated, retrieved, and deleted without directly exposing database operations to the Presentation or Business Layers.
- As a result, these architectures support future expansion such as digital wallets, robotic warehouses, smart-room automation, autonomous delivery vehicles, renewable-energy monitoring, and AI-based prediction systems without requiring major changes to the existing core system.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation